Lava Provider Guide

Field	Value
Document	`docs/guides/providers.md`
Revision	1
Last updated	2026-06-08
Status	Current
Scope	User-facing guide to every provider Lava ships, its auth, its capabilities, and how downloads work

> Every claim below is grounded in the real `*Descriptor.kt` and magnet-cache source files cited inline. Where a capability is declared on a descriptor but not yet exposed through the SDK, that gap is called out explicitly (no guessing — §11.4.6).

What a "provider" is in Lava

Lava is a multi-tracker client. Each supported site is a **provider** described by a `TrackerDescriptor` (the contract lives in `core/tracker/api/src/main/kotlin/lava/tracker/api/TrackerDescriptor.kt`). A descriptor declares:

- `trackerId` / `displayName` — the stable id and the label you see in the app.
- `baseUrls` — the primary site plus mirrors the SDK can fail over to.
- `capabilities` — the exact set of features the SDK will resolve for that provider. Per **Capability Honesty (constitutional clause §6.E)** a declared capability MUST map to a real, working feature — a provider never advertises something it cannot actually do.
- `authType` — how (or whether) you log in: `CAPTCHA_LOGIN`, `FORM_LOGIN`, or `NONE`.
- `verified` — whether the provider's end-to-end flow has passed a real-device Challenge Test (§6.G). `verified = false` means the provider is wired but its full user flow has a known gap (see the per-provider notes).

Capability matrix

Read directly from the six descriptor files. Legend: ✅ = declared and resolved through the SDK; ⚠️ = wired in code but **not** exposed via the descriptor (honest absence per §6.E); × = not applicable to that provider.

Capability	RuTracker	RuTor	Kinozal	NNM-Club	In Ar
Auth type	`CAPTCHA_LOGIN`	`FORM_LOGIN`	`FORM_LOGIN`	`FORM_LOGIN`	`NO`
SEARCH	✅	✅	✅	✅	✅
BROWSE	✅	✅	✅	✅	✅
FORUM	✅	×	×	⚠️ not wired	✅
TOPIC	✅	✅	✅	✅	✅
COMMENTS	✅	✅	✅	✅	×
FAVORITES	✅	×	×	⚠️ no endpoint	×
TORRENT_DOWNLOAD (`.torrent`)	✅	✅	✅	✅	× I or
MAGNET_LINK	✅	✅	✅	✅	×
RSS	×	✅	×	×	×
AUTH_REQUIRED	✅	✅ (optional)	✅	✅	×
Anonymous browse/ search	×	✅	×	×	✅ (ir
`verified` (Challenge-tested flow)	✅ true	✅ true	❌ false	❌ false	✅
Text encoding	Windows-1251	UTF-8	windows-1251	windows-1251	UT

Sources: `RuTrackerDescriptor.kt`, `RuTorDescriptor.kt`, `KinozalDescriptor.kt`, `NnmclubDescriptor.kt`, `ArchiveOrgDescriptor.kt`, `GutenbergDescriptor.kt` (all under `core/tracker/<id>/src/main/kotlin/lava/tracker/<id>/`).

Per-provider detail

RuTracker.org (`rutracker`)

- **What it is:** the largest Russian torrent tracker. Primary `https://rutracker.org`, with `rutracker.net` and `rutracker.cr` as failover mirrors.
- **Auth:** `CAPTCHA_LOGIN` — login can present a CAPTCHA you must solve. This is the only provider with CAPTCHA in its capability set.
- **What you can do:** search, browse forums (`FORUM`), open topics, read comments, keep favorites, and download both `.torrent` files and magnet links.
- **API support:** RuTracker is the one provider with a working route family in the `lava-api-go` backend today (`apiSupported = true`; see the `RuTrackerDescriptor.kt` Phase-1 hotfix note).
- **Verified:** yes — Challenge Tests C1, C2, C4, C5, C7, C8.
- **Note:** `UPLOAD` and `USER_PROFILE` were intentionally dropped from the descriptor because no SDK feature interface backs them (clause §6.E); the legacy upload/profile plumbing still ships but is not advertised.

RuTor.info (`rutor`)

- **What it is:** an anonymous-friendly Russian tracker. Mirrors include `rutor.info`, `rutor.is`, the `www.` variants, and an IPv6-only `6tor.org`.
- **Auth:** `FORM_LOGIN` (a plain form POST — no CAPTCHA). `AUTH_REQUIRED` is declared so the SDK can expose optional actions like adding a comment, but **search and browse work without logging in** (`supportsAnonymous = true`).
- **What you can do:** search, browse categories (no nested forum tree — `FORUM` is intentionally absent), open topics, read/add comments, RSS, and download `.torrent` + magnet.
- **Verified:** yes — Challenge Tests C1, C3, C4, C6, C7, C8.

Kinozal.tv (`kinozal`)

- **What it is:** a Russian movie/media tracker. Mirrors `kinozal.tv`, `kinozal.me`.
- **Auth:** `FORM_LOGIN`.
- **What you can do:** search, browse, open topics, read comments, download `.torrent` + magnet.
- **Verified: no** (`verified = false`). The descriptor records a known post-login navigation gap on the onboarding screen (`.lava-ci-evidence/sixth-law-incidents/2026-05-04-onboarding-navigation.json`). Treat Kinozal as wired-but-not-yet-fully-verified end to end.

NNM-Club (`nnmclub`)

- **What it is:** a Russian tracker with an embedded forum. Mirrors `nnmclub.to`, `nnm-club.me`.
- **Auth:** `FORM_LOGIN`.
- **What you can do:** search, browse, open topics, read comments, download `.torrent` + magnet. **`FORUM` is not wired** through the SDK surface yet, and **`FAVORITES` has no scraping endpoint** — both are honestly absent from the capability set (§6.E).
- **Verified: no** (`verified = false`) — same post-login navigation gap as Kinozal.

Internet Archive (`archiveorg`)

- **What it is:** a digital library, **not a torrent tracker** — `archive.org`, served over JSON + HTTP.
- **Auth:** `NONE` (implicitly anonymous).
- **What you can do:** search, browse, open topic metadata, and browse a forum surface. **No torrents and no magnets** — files are served over HTTP, so `TORRENT_DOWNLOAD` and `MAGNET_LINK` are deliberately absent.
- **Verified:** yes — Challenge Test C11 (continue → authorized main app).

Project Gutenberg (`gutenberg`)

- **What it is:** a free e-book library backed by the Gutendex JSON API (`https://gutendex.com`) — **not a torrent tracker**.
- **Auth:** `NONE` (public, unauthenticated API).
- **What you can do:** search, browse, open a book's topic page. It serves EPUB / plain-text / HTML e-books **over HTTP, not `.torrent`** — declaring `TORRENT_DOWNLOAD` would be a §6.E bluff, so it is absent.
- **Verified:** yes — Challenge Test C12.

How downloads work

Lava distinguishes two artifact kinds, by provider type.

Torrent providers (RuTracker, RuTor, Kinozal, NNM-Club)

These providers expose **two** download paths:

1. **`.torrent` file (`TORRENT_DOWNLOAD`).** Lava fetches the actual torrent file from the tracker's download endpoint and hands it off. The bytes are validated as a real bencoded torrent before use — see `core/common/src/main/kotlin/lava/common/torrent/TorrentFileValidator.kt` and `Bencode.kt`.

2. **Magnet link (`MAGNET_LINK`).** This is where the §6.E "Capability Honesty" design matters. The `DownloadableTracker.getMagnetLink` method is **synchronous** (non-`suspend`), and its contract is to return the magnet only when it is available **without** an extra HTTP fetch — otherwise `null` (an honest absence, never a fabricated value).

   To make that honest, Kinozal and NNM-Club use an in-memory, process-lifetime **magnet cache**:

   - `core/tracker/kinozal/src/main/kotlin/lava/tracker/kinozal/magnet/KinozalMagnetCache.kt`
   - `core/tracker/nnmclub/src/main/kotlin/lava/tracker/nnmclub/http/NnmclubMagnetCache.kt`

   The flow: when you **open a topic** (or, for NNM-Club, when a **search** result row is parsed) the production parser extracts the genuinely-present magnet and writes it into the cache (`put(topicId, magnet)`). The download feature later reads it back (`get(topicId)`). So **the magnet surfaces after you have viewed the topic/search** — at that point the synchronous lookup returns the real magnet with no hidden round-trip. If you ask for a magnet for a topic you have never opened, the cache returns `null` rather than inventing one. Both caches are `@Singleton`, backed by a thread-safe `ConcurrentHashMap`, so the topic feature and the download feature share one instance per process. Magnet links are validated by `core/common/.../torrent/MagnetLinkValidator.kt`.

Library providers (Internet Archive, Project Gutenberg)

These are **HTTP downloads, not torrents.** Internet Archive serves files directly; Project Gutenberg serves EPUB / plain-text / HTML e-books over HTTP via Gutendex. Neither declares `TORRENT_DOWNLOAD` or `MAGNET_LINK`. (Note from `GutenbergDescriptor.kt`: an HTTP-download implementation exists in code but is not exposed through `getFeature()` because `TrackerCapability` has no `HTTP_DOWNLOAD` value today — an honest not-yet-wired state.)

Quick reference: which providers need credentials?

Need to log in?	Providers
Yes (always)	RuTracker (CAPTCHA), Kinozal, NNM-Club
Optional (anonymous search/ browse works)	RuTor
No (anonymous)	Internet Archive, Project Gutenberg

For how to enter and store credentials, see the in-app provider login flow; real credentials live only in the gitignored `.env` (§6.H) and are never committed.